KIRA AI Incident Report

Cluster Summary

The Kubernetes cluster is currently exhibiting signs of **silent degradation and configuration drift**. While the Kubernetes API reports 8 pods in a `Running` phase, Prometheus telemetry indicates only **5 active pods**. This critical discrepancy indicates that 3 pods are either failing readiness probes (and are therefore not registered as active endpoints) or are stuck in rapid restart loops (`CrashLoopBackOff` where the phase briefly shows as `Running`).

Furthermore, there is a clear service duplication anomaly: both `orders-5f6cf55446-cg2qv` and `order-service-5b7987bdd-v2t2c` are running concurrently. This suggests a deployment pipeline failure or uncleaned legacy resources, which is likely causing database lock contention on `boutique-postgres-0` and resource exhaustion.

Key Findings

1. **Pod State Discrepancy (37.5% Drift):**
* Kubernetes API Pod Count: **8**
* Prometheus `active_pods` Metric: **5**
* **Impact:** Three pods are non-functional, unready, or unable to serve traffic despite being in a `Running` phase.
2. **Duplicate Service Deployment:**
* Both `orders-*` and `order-service-*` exist in the namespace.
* This redundancy splits traffic (if they share selector labels) or wastes database connections to `boutique-postgres-0`.
3. **Elevated Memory Footprint:**
* Memory usage is at **73%** while CPU usage is at **52%**. This high memory-to-CPU ratio indicates potential memory leaks or resource-limit starvation from the duplicate deployments.
4. **Log Analysis:**
* Loki logs confirm successful initialization of the *orders* service on port `3005` and connection to PostgreSQL. However, we have zero logs from the duplicate `order-service`, pointing to it as a primary suspect for the unready/failing state.

Severity Assessment

Severity: **SEV-2 (Major Degradation / High Risk of Split-Brain)**

Why:
* **Capacity Loss:** 3 out of 8 pods are failing silently, leaving the cluster operating at reduced capacity.
* **Data Integrity Risk:** Two distinct ordering deployments (`orders` and `order-service`) accessing the same database (`boutique-postgres-0`) can cause schema locks, race conditions, or duplicate writes.
* **Resource Exhaustion Risk:** At 73% memory utilization, any traffic spike or another microservice restart could trigger Node-level Out-of-Memory (OOM) killer events.

Root Cause Analysis

1. **Deployment Pipeline Drift:**
An incomplete Helm upgrade, ArgoCD sync error, or manual intervention left a legacy deployment (`orders` or `order-service`) active.
2. **Readiness Probe Failures:**
The 3 missing "active" pods are likely failing their `/healthz` or `/ready` endpoints. This is highly common when a database connection pool is exhausted due to duplicate services hogging PostgreSQL connections.
3. **Database Connection Exhaustion:**
`boutique-postgres-0` has a limited connection pool. Having two separate deployments (`orders` and `order-service`) scaling independently will exhaust Postgres `max_connections`, causing newly spawned pods to fail their readiness probes and drop out of the "active" pool.

Recommended Remediation

Phase 1: Immediate Triage
1. **Identify and Terminate Legacy Deployment:** Determine which deployment (`orders` vs. `order-service`) is the correct active production version. Terminate the deprecated one.
2. **Free Database Connections:** Deleting the duplicated deployment will immediately free up TCP sockets and database connection slots on `boutique-postgres-0`.
3. **Force Restart Failing Pods:** Identify the 3 unready pods and perform a rolling restart to force them to re-establish clean connections.

Phase 2: Configuration Alignment
1. **Update Service Selectors:** Ensure Kubernetes Services and Ingress point exclusively to the correct backend pods.

Preventative Measures

1. **Implement Strict GitOps (ArgoCD/Flux):** Enable auto-pruning in your CD pipelines to ensure old/renamed deployments are deleted automatically.
2. **Refine Readiness/Liveness Probes:** Ensure all pods have properly configured TCP/HTTP probes with appropriate timeouts and failure thresholds.
3. **Set Pod Resource Limits:** Apply strict CPU/Memory requests and limits on all workloads to prevent a single leaking container from taking down the node (mitigating the 73% memory threat).
4. **Configure Database Connection Pooling:** Implement a database proxy like **PgBouncer** to handle scaling connection requests gracefully.

Exact kubectl Commands to diagnose and fix the issue

1. Diagnose the Pod discrepancy & identify the 3 unready pods
```bash
# List all pods showing Ready status columns (e.g., 0/1 indicate unready pods)
kubectl get pods -n default

# Find pods with non-zero restart counts (indicating CrashLoopBackOff)
kubectl get pods -n default -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,RE
ADY:.status.containerStatuses[0].ready,RESTARTS:.status.containerStatuses[0].restartCount
```

2. Inspect Probe failures & Cluster Events
```bash
# Search for Warning events, specifically looking for Failed liveness/readiness probes
kubectl get events -n default --field-selector type=Warning --sort-by='.metadata.creationTimestamp'
```

3. Check logs of both order services
```bash
# Tail logs of the "orders" pod
kubectl logs -l app=orders -n default --tail=50

# Tail logs of the "order-service" pod to see if it is failing to connect to Postgres
kubectl logs -l app=order-service -n default --tail=50
```

4. Investigate Database Connection State
```bash
# Execute into Postgres to check the active connection count
kubectl exec -it boutique-postgres-0 -n default -- psql -U postgres -c "SELECT count(*), state
FROM pg_stat_activity GROUP BY state;"
```

5. Fix: Delete the legacy deployment
(Assuming `orders` is the legacy service, and `order-service` is the standard current service)
```bash
# Delete the deprecated deployment to free up system resources and DB connections
kubectl delete deployment orders -n default
```

6. Force a rolling restart of the remaining healthy deployments to clear stale connections
```bash
# Restart the active order service to clean up its connection state
kubectl rollout restart deployment order-service -n default

# Monitor the rollout and verify that active_pods returns to 100% target match
kubectl get pods -n default -w
```